

NAWIGACJA ROBOTÓW HUMANOIDALNYCH

WYKŁAD 5

VLM i VLA w nawigacji
robota humanoidalnego

dr inż. Mateusz Pomianek



VLM

VLA

Sem. nav.

ROS 2 + AI

Plan wykładu

Sekcja 1

VLM - architektura, modele CLIP/LLaVA/GPT-4V, grounding, zero-shot reasoning w robotyce

Sekcja 2

VLA - modele RT-2, OpenVLA, $\pi 0$, Octo; architektura i generowanie akcji z języka i wizji

Sekcja 3

Praktyczne zastosowania: nawigacja semantyczna, instruction following, integracja z ROS 2

Kody Python

Zapytanie VLM o scenę, grounding w mapie, router VLM→planner, pętla VLA w symulacji

01

SEKCJA

VLM - modele wizyjno-językowe

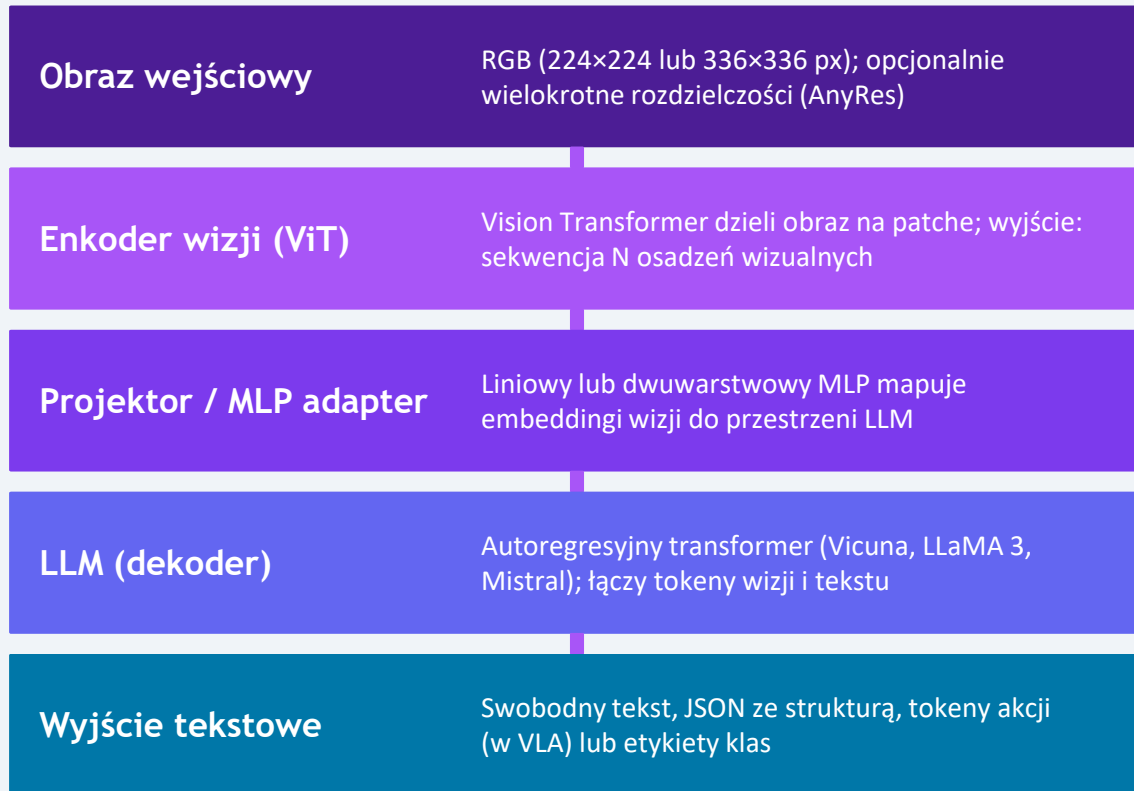
*Architektura, kluczowe modele, grounding
i zastosowania w robotyce*



Czym jest Vision-Language Model (VLM)?

- **VLM** to model uczenia głębokiego łączący kodowanie obrazu i tekstu we wspólnej przestrzeni reprezentacji
- **Zadania:** opis obrazu (captioning), VQA (visual question answering), wykrywanie obiektów z opisu, rozumowanie przestrzenne
- **Architektura typowa:** enkoder wizji (ViT lub CNN) + projektor + duży model językowy (LLM)
- **Trening:** pretraining na miliardach par obraz-tekst z internetu (LAION, CC12M, WebLI...)
- **Fine-tuning:** dostrojenie na zadaniach robotycznych z parami (obs, opis, akcja)
- **Przykłady:** CLIP (OpenAI), LLaVA-1.6, InstructBLIP, GPT-4o Vision, Gemini 1.5 Pro, Qwen-VL
- **Kluczowa własność:** zero-shot generalization - rozumienie scen niewidzianych podczas treningu

Architektura VLM - od pikseli do tokenów



Kluczowe cechy

- Freeze ViT → trenuj tylko projektor + LLM (LLaVA)
- Jeden forward pass przez cały stos
- Kontekst: typowo 4 096-128 000 tokenów
- Parametry: 7B-72B (edge: 3B-7B)
- Latencja GPU: 100-800 ms / zapytanie
- Kwantyzacja 4-bit na embedded GPU

CLIP - fundament grounded perception

Idea CLIP (OpenAI, 2021)

- **Trening kontrastowy:** 400M par (obraz, opis) z internetu
- **Dwa enkodery:** Image Encoder (ViT-L) + Text Encoder (transformer)
- **Cel:** maksymalizacja podobieństwa $\cos.$ dla pasujących par
- **Wynik:** wspólna przestrzeń embeddingów o wymiarze 512-1024
- **Możliwość zero-shot klasyfikacji:** obraz vs. opisy klas
- Baza dla LLaVA, DALL-E, Flamingo, CogVLM

CLIP w nawigacji humanoida

- **Lokalizacja semantyczna:** podobieństwo embedding (klatka) vs. embedding (opis celu)
- Np. "znajdź niebieskie drzwi" → szukaj w mapie klatek z max similarity
- **Object grounding:** ranking regionów ROI vs. zapytanie językowe
- **Działanie w czasie rzeczywistym:** CLIP ViT-B/16 ~10 ms na GPU
- **Ograniczenie:** brak rozumowania wieloetapowego; tylko matching

LLaVA i Visual Question Answering w robotyce

- **LLaVA (Large Language and Vision Assistant):** CLIP ViT + 2-layer MLP projektor + LLaMA/Vicuna
- **Trening dwuetapowy:** (1) freeze ViT, ucz projektor; (2) fine-tune projektor + LLM na danych instrukcji
- **VQA dla robota:** robot zadaje pytanie o obraz z kamery; VLM odpowiada w języku naturalnym
- **Przykłady pytań VQA:** "Czy drzwi są otwarte?", "Ile osób stoi przy windzie?", "Który korytarz jest wolny?"
- **Odpowiedź strukturalna:** prompt wymuszający JSON: {"door_open": true, "people_count": 2, "clear_path": "left"}
- **LLaVA-1.6 (34B):** rozumowanie przestrzenne, zliczanie, rozróżnianie lewej/prawej strony
- **Wersje on-device:** LLaVA-Phi-3-mini (3,8B) - możliwa na Jetson Orin AGX (~200 ms / klatka)

Grounding semantyczny - od słów do lokalizacji

1

Opis celu

Użytkownik lub system misji podaje opis: "idź do stolika z kawą przy oknie"

2

Feature extraction

VLM (LLaVA lub CLIP) koduje klatki z kamery; VLM opisuje każdy kandydujący region mapy

3

Matching

Cosinus similarity lub LLM scoring między embeddingiem opisu, a embeddingami regionów

4

Lokalizacja na mapie

Region z najwyższym score → jego współrzędne (x,y) na mapie → cel dla A*

Zero-shot vs. few-shot reasoning w nawigacji

Zero-shot

- Model nigdy nie widział danej sceny ani zadania podczas treningu
- Polecenie: "idź do drukarki" - VLM rozumie bez przykładów
- Działa gdy obiekt jest w słowniku treningu VLM (>1B tokenów tekstu)
- Słabszy dla specyficznego sprzętu laboratoryjnego lub lokalnych obiektów
- Praktyka: GPT-4o opisuje scenę i wskazuje cel nawigacyjny

Few-shot (in-context learning)

- 2-5 przykładów par (obraz, opis, akcja) w prompcie kontekstowym
- Nie wymaga fine-tuningu - szybkie dostosowanie do nowego środowiska
- Np. 3 zdjęcia korytarza ze strzałkami + pytanie → kierunek ruchu
- Kluczowe dla robotów przemysłowych z niestandardowymi obiektami
- Ograniczenie: długi kontekst = wyższy koszt inferowania i latencja

Zapytanie VLM o stan sceny (Anthropic / OpenAI API)

```
import base64, httpx, json

def query_vlm_scene(image_bgr, question: str,
                    model='claude-sonnet-4-6') -> dict:
    """Wysyla klatke kamery do VLM i zwraca odpowiedz JSON."""
    import cv2
    _, buf = cv2.imencode('.jpg', image_bgr, [cv2.IMWRITE_JPEG_QUALITY, 85])
    b64 = base64.b64encode(buf).decode()
    payload = {
        "model": model, "max_tokens": 256,
        "messages": [{ "role": "user", "content": [
            { "type": "image", "source": { "type": "base64",
            "media_type": "image/jpeg", "data": b64 }},
            { "type": "text", "text": f'Odpowiedz TYLKO JSON. {question}" }
        ]}]
    }
    r = httpx.post('https://api.anthropic.com/v1/messages',
                  json=payload, headers={'x-api-key': API_KEY,
                  'anthropic-version': '2023-06-01'}, timeout=5.0)
    return json.loads(r.json()['content'])[0]['text'])
```

Pytanie np.: "Czy droga do przodu jest wolna? **Odpowiedz:** {"clear": true/false, "obstacle": string}."

02

SEKCJA

VLA - modele wizyjno-językowe- akcyjne

*RT-2, OpenVLA, $\pi 0$, Octo, architektura i
generowanie akcji*



Czym jest Vision-Language-Action Model (VLA)?

- VLA rozszerza VLM o głowicę akcji (action head): model generuje nie tylko tekst, lecz tokeny ruchu
- Wejście: obraz (lub sekwencja klatek) + polecenie językowe; wyjście: wektor akcji lub tokeny dyskretne
- RT-2 (Google DeepMind, 2023): fine-tuning PaLI-X (55B) i PaLM-E na danych robotycznych
- OpenVLA (Stanford, 2024): open-source; Prismatic VLM + 7B LLaMA; tokeny akcji jako tekst
- $\pi 0$ (Physical Intelligence, 2024): diffusion policy head dla ciągłych akcji; duże słownictwo gestów
- Octo (Berkeley, 2024): transformer-based; multi-task; przyjmuje różne modalności wejściowe
- Kluczowa różnica od klasycznej polityki RL: VLA używa wiedzy językowej do generalizacji zero-shot

Architektura VLA, od obserwacji do akcji



Kluczowe cechy

- Częstość wyjść: 3-10 Hz (ograniczona przez LLM)
- Buforowanie akcji: temporal ensemble lub action chunking
- Fine-tuning: ~100-10 000 demonstracji
- Imitation learning + RL fine-tuning
- Deployment: Jetson AGX Orin (7B q4) $\approx$ 3 Hz
- Bezpieczeństwo: clip akcji + velocity guard

Przegląd kluczowych modeli VLA

RT-2 (Google, 2023)

- Base: PaLI-X 55B lub PaLM-E 562B
- Tokeny akcji: dyskretne (256 *bin/oś*)
- Co-fine-tuning na Web + Robot data
- Generalizacja zero-shot do nowych obiektów
- Proprietary; działa na TPU v4
- Wykazuje chain-of-thought reasoning

OpenVLA (Stanford, 2024)

- Base: Prismatic (*SigLIP + DINOv2*) + LLaMA 7B
- Open source: wagi na HuggingFace
- Fine-tuning na Open-X-Embodiment (~970 k ep.)
- Akcje jako tekst: "move_left 0.3"
- LoRA fine-tuning w <24 h na 1× A100
- Dobrze udokumentowane API

$\pi 0$ / Octo (2024)

- $\pi 0$: diffusion action head (*ciągłe akcje*)
- Wspiera dexterous manipulation
- Octo: multi-task transformer policy
- Modular: różne enkodery wizji
- Open-X-Embodiment benchmark SoTA
- Integracja z ROS 2 przez action server

Tokenizacja akcji - jak VLA generuje ruch

- **Dyskretna tokenizacja:** zakres każdej osi ruchu (np. $\pm 0,05$ m/krok) dzielony na 256 równych binów
- **Każda oś akcji** $\rightarrow$ jeden token z alfabetu $[0...255]$; cały wektor akcji = K tokenów po sobie
- **Dekodowanie:** token_id $\rightarrow$ wartość ciągła przez de-binning + ewentualne odsumowanie
- **Action chunking:** VLA generuje N kolejnych kroków akcji naraz (np. $N=10$); redukuje latencję
- **Temporal ensemble:** ważona suma K ostatnich prognoz dla bieżącego kroku - wygładzenie ruchu
- **Diffusion policy (π_0):** akcja jako wynik odwróconego procesu dyfuzji; ciągła, wielomodalna
- **Hybrid:** tokeny dyskretne dla nawigacji (v, ω) + ciągłe diffusion dla manipulacji dłońmi

Fine-tuning VLA na danych robotycznych

Zbieranie danych demonstracji

- **Teleoperation:** operator steruje robotem; zapis (obs, action) w ~ 10 Hz
- **Symulacja:** generowanie danych w MuJoCo / Isaac Sim (szybko, bez ryzyka)
- **Play data:** swobodne zbieranie bez konkretnego zadania; labels posta hoc
- **Liczba demonstracji:** 100-10 000 epizodów zależy od złożoności zadania
- **Format:** HDF5 lub RLDS; zgodność z Open-X-Embodiment dataset

Strategie fine-tuningu

- **Full fine-tuning:** wszystkie wagi; potrzebny duży GPU (≥ 80 GB VRAM)
- **LoRA (Low-Rank Adaptation):** trainable rank-r matrices; 16-32 GB GPU
- **Adapter modules:** dodatkowe warstwy między blokami transformera
- **Co-training:** robot data + Web data $\rightarrow$ zapobiega catastrophic forgetting
- **Ewaluacja:** success rate na held-out zadaniach w symulacji i na robocie

Integracja OpenVLA w pętli sterowania (HuggingFace)

```
from transformers import AutoModelForVision2Seq, AutoProcessor
import torch, numpy as np

processor = AutoProcessor.from_pretrained('openvla/openvla-7b',
                                         trust_remote_code=True)

model = AutoModelForVision2Seq.from_pretrained(
    'openvla/openvla-7b', torch_dtype=torch.bfloat16,
    low_cpu_mem_usage=True, trust_remote_code=True).cuda()

def get_action(image_rgb: np.ndarray, instruction: str) -> np.ndarray:
    """Zwraca wektor akcji [dx, dy, dz, drx, dry, drz, gripper]."""
    prompt = f'In: What action should the robot take to {instruction}?\nOut:'
    inputs = processor(prompt, image_rgb).to('cuda:0', torch.bfloat16)
    with torch.inference_mode():
        action = model.predict_action(**inputs, unnorm_key='bridge_orig',
                                     do_sample=False)
    return action # numpy (7,) [m, m, m, rad, rad, rad, bool]
```

Opis: OpenVLA zwraca 7-DoF akcję w jednostkach metrycznych; unnorm_key określa skalowanie do danego robota.

Latencja i deployment VLA na robocie

Cloud GPU (A100/H100)

Latencja ~80-150 ms; konieczna łączność sieciowa; nieakceptowalne dla dynamicznych zadań

Edge GPU (Jetson Orin)

Latencja 300-600 ms dla 7B q4; wystarczające dla nawigacji; niezależność od chmury

Action chunking

Generuj $N=10$ kroków naraz; wykonuj lokalnie; VLA odpytywana co $N \times dt$ ms $\rightarrow$ efektywnie 10-30 Hz

Distillacja / pruning

Nauczyciel 34B $\rightarrow$ student 3B; zachowanie 85-90% skuteczności przy 5 $\times$ szybszej inferencji

Kwantyzacja 4-bit

AWQ lub GPTQ: 7B model zajmuje ~4 GB VRAM; minimalna utrata jakości akcji ($< 3\%$ success rate)

Speculative decoding

Draft model (0,5B) proponuje tokeny; duży model weryfikuje; 2-3 $\times$ przyspieszenie bez utraty jakości

03

SEKCJA

Praktyczne zastosowania w nawigacji humanoida

*Nawigacja semantyczna, instruction following,
integracja z ROS 2, ograniczenia*



Nawigacja semantyczna z VLM

- **Klasyczna nawigacja:** cel = współrzędne (x, y); robot nie rozumie sensu celu
- **Nawigacja semantyczna:** cel = opis językowy "znajdź salę z projekcją slajdów" lub "idź do kuchni"
- **Etap offline: budowa mapy semantycznej:** robot objeżdża budynek; VLM opisuje każdą pozycję
- **Etap online: lokalizacja celu:** embedding(opis celu) vs. embeddingi pozycji w mapie → argmax sim
- **CLIP-Fields (2022):** neural field z embeddingami CLIP w każdym wokselu mapy 3D
- **SQA3D / EQA (Embodied QA):** robot odpowiada na pytania o przestrzeń, poruszając się po środowisku
- **ConceptFusion (2023):** mapa z deskryptorami CLIP + SAM na chmurze punktów - open-vocab grounding 3D

Pipeline instruction following - od polecenia do akcji

1

**Polecenie
użytkownika**

"Przynieś mi kawę z kuchni
na 2. piętrze" - tekst lub
mowa (ASR → tekst)

2

LLM task planner

Duży LLM (GPT-4o) rozkłada
polecenie na podrzędne
cele: navigate → pick →
navigate → place

3

**VLM scene
grounding**

CLIP lub LLaVA lokalizuje
obiekty ("kawa", "kuchnia")
w mapie semantycznej

4

Nav2 + footstep

Klasyczna nawigacja z A* do
każdego podzadania;
footstep planner dla
precyzyjnych ruchów

Grounding semantyczny w mapie topologicznej

```
import numpy as np
from transformers import CLIPModel, CLIPProcessor
import torch

class SemanticMap:
    def __init__(self, model_name='openai/clip-vit-base-patch16'):
        self.model = CLIPModel.from_pretrained(model_name).eval().cuda()
        self.proc = CLIPProcessor.from_pretrained(model_name)
        self.nodes = [] # lista (x, y, emb) wezlow topologicznych

    def add_node(self, x, y, image_rgb):
        inp = self.proc(images=image_rgb, return_tensors='pt').to('cuda')
        with torch.no_grad():
            emb = self.model.get_image_features(**inp)
            emb = emb / emb.norm(dim=-1, keepdim=True) # normalizacja L2
        self.nodes.append((x, y, emb.cpu().numpy()))

    def find_goal(self, query_text: str) -> tuple:
        inp = self.proc(text=[query_text], return_tensors='pt').to('cuda')
        with torch.no_grad():
            t_emb = self.model.get_text_features(**inp)
            t_emb = (t_emb / t_emb.norm()).cpu().numpy()
            scores = [float(t_emb @ n[2]).T for n in self.nodes]
            best = int(np.argmax(scores))
            return self.nodes[best][0], self.nodes[best][1] # (x, y)
```

Opis: add_node() buduje mapę podczas eksploracji; find_goal() lokalizuje cel z opisu w ~2 ms

Router: VLM ocenia scenę i wywołuje właściwy planistę

```
import json

ROUTING_PROMPT = '''
Analizujesz obraz z kamery robota humanoidalnego.
Zwroc TYLKO JSON z polami:
    "situation": "normal"|"obstacle_close"|"stairs"|"person_blocking",
    "confidence": 0.0-1.0,
    "recommended_action": "continue"|"replan"|"slow_down"|"stop"|"ask_human"
'''

def vlm_route(image_rgb, nav_node) -> str:
    result = query_vlm_scene(image_rgb, ROUTING_PROMPT) # z Kodu 1
    situation = result.get('situation', 'normal')
    confidence = result.get('confidence', 0.0)
    action = result.get('recommended_action', 'continue')
    if confidence < 0.6:
        return 'continue' # niepewny wynik - jedz dalej
    if action == 'replan':
        nav_node.trigger_replan() # sygnał do Nav2
    elif action == 'stop':
        nav_node.emergency_stop()
    return action
```

Opis: Router wywołuje VLM co ~500 ms (asynchronicznie); próg confidence=0,6 filtruje halucynacje modelu

Integracja VLM/VLA z ROS 2 - wzorzec asynchroniczny

- **VLM działa zbyt wolno** (>100 ms) dla synchronicznej pętli Nav2 (50 Hz) - konieczna asynchroniczność
- **Wzorzec:** dedykowany node `/vlm_node` subskrybuje `/camera/image_raw`, publikuje `/vlm_observations`
- **Buffer:** `/vlm_observations` to topic z QoS `BEST_EFFORT`, `depth=1` - zawsze najnowsza interpretacja
- **Nav2 BehaviorTree:** VLM jako zewnętrzny condition node - sprawdza buffer i blokuje w razie "stop"
- **Action server:** `/vlm_ground_goal` [`text_goal` → `PoseStamped`] - lokalizacja semantyczna na żądanie
- **Lifecycle node:** VLM node uruchamiany/zatrzymywany przez Nav2 state machine (oszczędność GPU)
- **Monitoring:** `/vlm_diagnostics` - latencja, liczba halucynacji wykrytych przez sanity check

Ograniczenia i wyzwania VLM/VLA w robotyce

Problemy techniczne

- **Halucynacje:** VLM "widzi" obiekty których nie ma lub fałszywie ocenia odległości
- **Latencja:** 100-800 ms; nieakceptowalne przy szybkim ruchu ($>0,5$ m/s)
- **Dryft kontekstu:** długie konwersacje tracą koherencję opisu środowiska
- **Sim-to-real gap:** model trenowany na zdjęciach z internetu słabo generalizuje na obraz fisheye robota
- **Koszt zasobów:** 7B model wymaga 16 GB VRAM; 34B - 70+ GB
- **Niespójność czasowa:** opis sceny w klatce t nie zawsze pasuje do pozycji w klatce $t+5$

Problemy bezpieczeństwa i niezawodności

- **Prompt injection:** złośliwy tekst w scenie może manipulować planistą
- **Brak gwarancji:** VLM nie daje formalnych gwarancji bezpieczeństwa (bez dowodów poprawności)
- **Fail-open vs fail-safe:** przy błędzie VLM robot musi zatrzymać się, nie kontynuować
- **Weryfikacja krzyżowa:** wyniki VLM powinny być sanity-checked przez klasyczne sensory (LiDAR)
- **Biasy modeli:** VLM może inaczej reagować na różnych ludzi ze względu na biasy danych treningowych

Przykłady wdrożeń VLM/VLA w humanoidach (2023-2025)

Figure 01 + OpenAI GPT-4o

Humanoid Figure prowadzi rozmowę o środowisku i wykonuje zadania na podstawie poleceń języka naturalnego

Unitree H1 + VLM planner

H1 z LLaVA jako scene understander: autonomiczne nawigowanie w biurze do opisanych celów

Boston Dynamics Spot + GPT-4

Spot opisuje to co "widzi,, i odpowiada na pytania; tryb VQA do inspekcji środowiska przemysłowego

RT-2 (Everyday Robots)

Robot Google wykonuje zadania pick-place z poleceń języka naturalnego; SoTA zero-shot na RT-Benchmark

$\pi 0$ (Physical Intelligence)

Humanoidalny robot kuchenny wykonuje złożone zadania (owijanie naleśników) sterowany VLA $\pi 0$

LEROBOT (HuggingFace 2024)

Open-source platforma RL+IL+VLA; demonstracje na SO-100/LeKiwi; wspólnota ~50k gwiazdek GitHub

Najlepsze praktyki inżynierskie - VLM/VLA w produkcji

- **Separacja pętli:** pętla bezpieczeństwa (1 kHz, deterministyczna) zawsze nadrzędna wobec VLM (2-5 Hz)
- **Sanity check akcji:** każda akcja VLA clampowana do fizycznie osiągalnego zakresu; ZMP sprawdzany przed wykonaniem
- **Confidence threshold:** odrzucaj wyniki VLM z confidence $< 0,7$; fallback do klasycznego planisty
- **Prompt engineering:** jasny, deterministyczny prompt z wymuszeniem formatu JSON + przykładem (few-shot)
- **Logging i replay:** zapis każdego zapytania VLM + odpowiedzi do MCAP/rosbag2 - postmortem po incydencie
- **Graceful degradation:** gdy VLM niedostępny $\rightarrow$ robot przechodzi w tryb klasycznej nawigacji metrycznej
- **Testy regresji:** automatyczny benchmark 50 zapytań VQA na obrazach referencyjnych przy każdej zmianie modelu

Kierunki badań i przyszłość VLM/VLA w nawigacji

World models

VLA z modelem świata (RSSM, JEPA): robot "wyobraża" sobie konsekwencje akcji przed ich wykonaniem

Embodied foundation models

Jeden model dla percepcji, planowania i sterowania; Gato, UniSim, UniPi - unifikacja wszystkich warstw

Continual learning

VLA aktualizuje się online z nowych demonstracji bez catastrophic forgetting; kluczowe dla wdrożeń

Multiagent VLM

Kilka robotów współdzielących mapę semantyczną; VLM jako wspólny language interface dla roju

Certifiable safety

Formalna weryfikacja wyjść VLA (conformal prediction, runtime monitors) dla środowisk krytycznych

Hardware acceleration

Dedykowane NPU dla transformerów (Qualcomm AI 100 Ultra, NVIDIA Thor); latencja <50 ms dla 7B on-device

PODSUMOWANIE

- 01 **VLM** (CLIP, LLaVA, GPT-4V) wnoszą **rozumienie języka naturalnego** i **semantyki sceny** do systemu nawigacji.
- 02 VLA (RT-2, OpenVLA, $\pi 0$) **zamykają pętlę**: obserwacja wizualna + polecenie językowe → akcja robota.
- 03 **Grounding semantyczny** pozwala nawigować do celu opisanego słowami bez uprzedniego uczenia mapy etykiet.
- 04 **Latencja VLM** (>100 ms) wymaga buforowania i asynchronicznego wywoływania względem pętli sterowania.
- 05 **Weryfikacja bezpieczeństwa** i sanityzacja wyjść VLA są obowiązkowe przed przekazaniem do WBC i aktuatorów.

Zadanie projektowe i pytania problemowe

1. Dlaczego pętla sterowania (1 kHz) i pętla VLM (2-5 Hz) muszą być rozdzielone? Narysuj schemat asynchronicznej architektury.
2. Jakie zagrożenia niesie prompt injection dla robota sterowanego VLM w środowisku z ludźmi? Zaproponuj mechanizm obrony.

Zadanie: zaimplementuj klasę SemanticMap (patrz Kod 3), zbierz embeddingi dla 10 pozycji w symulowanym środowisku (dowolny zbiór zdjęć) i sprawdź, które zapytania języka naturalnego są poprawnie rozwiązywane, a które nie.

Porównaj OpenVLA i klasyczny sterownik PID dla zadania "idź 2 m do przodu":
jakie są różnice w latencji, niezawodności i możliwości generalizacji?

Bonus: skonfiguruj pipeline LLaVA-1.6 (3B) + Nav2 w symulatorze Gazebo i zademonstruj nawigację do celu podanego opisem słownym.